

Open-FDD Documentation

Generated by scripts/build_docs_pdf.py

June 6, 2026

- 1 Home
- 2 Open-FDD
 - 2.1 What you get
 - 2.2 Two paths
 - 2.2.1 Try it with Docker (recommended)
 - 2.2.2 Develop locally
 - 2.3 Distribution
 - 2.4 License
- 3 Run with Docker images
- 4 Run with Docker images
 - 4.1 Images (GHCR)
 - 4.2 Option A — Ansible deploy (edge VM)
 - 4.3 Option B — Local trial (compose on one machine)
 - 4.4 Required configuration
 - 4.5 Start / stop
 - 4.6 After first deploy
- 5 First login and health check
- 6 First login and health check
 - 6.1 Health endpoints
 - 6.2 Login
 - 6.3 Smoke checklist
 - 6.4 Troubleshooting
- 7 Quick Start
- 8 Quick Start
- 9 Updating the stack
- 10 Updating the stack
 - 10.1 Path A — SSH on the live VM (minimal deploy folder)
 - 10.2 Path B — Ansible from control machine (inventory)
 - 10.3 Full redeploy
 - 10.4 Verify after upgrade
 - 10.5 Rollback
- 11 Local development
- 12 Local development
 - 12.1 Clone and auth
 - 12.2 Start stack
 - 12.3 BACnet bench overlay (optional)
 - 12.4 Docs preview

- 13 Build Docker images
- 14 Build Docker images
 - 14.1 Local build
 - 14.2 Publish to GHCR (maintainers)
- 15 Developer Guide
- 16 Developer Guide
- 17 Tests and CI
- 18 Tests and CI
 - 18.1 Local tests
 - 18.2 CI (GitHub Actions)
 - 18.3 Docs build (same as CI)
- 19 Contributing
- 20 Contributing
 - 20.1 Expectations
 - 20.2 Pull request checklist
 - 20.3 Repository layout (short)
 - 20.4 Security issues
 - 20.5 Agent-assisted development
- 21 System overview
- 22 System overview
 - 22.1 Components
 - 22.2 Optional PyPI-only path
- 23 Arrow recipes
- 24 Arrow recipes (default)
 - 24.1 Simple threshold
 - 24.2 Supply air temp high
 - 24.3 Fan commanded but no airflow
 - 24.4 Economizer not opening
 - 24.5 Heating and cooling simultaneously
 - 24.6 Sensor out of range
- 25 Data flow
- 26 Data flow
 - 26.1 Telemetry ingest
 - 26.2 Rule evaluation
 - 26.3 Model sync
 - 26.4 Retention
- 27 Python recipes (Arrow)
- 28 Python recipes (Arrow)
- 29 YAML recipes
- 30 YAML recipes
 - 30.1 Minimal rule file
 - 30.2 When to use YAML vs Python Rule Lab
- 31 Containers
- 32 Containers
- 33 Architecture
- 34 Architecture
- 35 Rule Cookbook
- 36 Rule Cookbook
- 37 Windowing & debugging
- 38 Windowing & debugging
 - 38.1 Rolling windows (Arrow)
 - 38.2 Rule Lab console

39	Dashboard overview
40	Dashboard overview
40.1	Primary areas
40.2	Live updates
40.3	Roles
41	Auth and roles
42	Auth and roles
42.1	Production (LAN / edge)
42.2	Localhost dev
43	Rule Lab
44	Rule Lab
45	Model workflow
46	Model workflow
46.1	Brick model
46.2	Typical integrator flow
47	Operator Bridge
48	Operator Bridge
49	Network setup
50	Network setup
50.1	Bind address
50.2	Discovery range
50.3	Verify
51	Discover and read
52	Discover and read
52.1	Operator workflow
53	Polling
54	Polling
54.1	Enable on edge
54.2	Health
55	Write safety
56	Write safety
56.1	Defaults
56.2	Enable writes (commission role)
56.3	Operator rules
57	BACnet
58	BACnet
59	Convention
60	Fault code convention
60.1	Format
60.2	Categories
60.3	Severity
60.4	Linking rules
60.5	Cookbook mapping
61	AHU faults
62	AHU faults
63	VAV faults
64	VAV faults
65	RTU faults
66	RTU faults
67	Sensor and data quality
68	Sensor and data quality
68.1	Communication quality

- 68.2 Alarm console quality
- 69 Fault Codes
- 70 Fault Codes
- 71 Live site update (SSH)
- 72 Live site update (SSH)
 - 72.1 Concepts (plain language)
 - 72.2 Before you start
 - 72.3 1. Backup workspace/
 - 72.4 2. Verify new images exist on GHCR
 - 72.5 3. Update the React UI bundle (required for dashboard changes)
 - 72.6 4. Update docker-compose.yml and recreate containers
 - 72.7 5. On-VM smoke checks
 - 72.7.1 Arrow / FDD rules (Open-FDD 3.0+)
 - 72.8 6. Control-machine insurance check
 - 72.9 7. Browser validation
 - 72.10 Rollback
 - 72.11 Alternative: Ansible image-only upgrade
- 73 LAN hardening
- 74 LAN hardening
 - 74.1 Deployment modes
 - 74.2 Network exposure
 - 74.3 Rule Lab caution
- 75 Secrets and auth
- 76 Secrets and auth
 - 76.1 Files (gitignored)
 - 76.2 Required variables (production)
 - 76.3 GHCR pulls
 - 76.4 Backup
- 77 BACnet write allowlists
- 78 BACnet write allowlists
- 79 Operations
- 80 Operations
- 81 Security
- 82 Security
- 83 API routes
- 84 API routes
 - 84.1 Health & audit
 - 84.2 Auth
 - 84.3 Rules & FDD
 - 84.4 Rule Lab playground
 - 84.5 BRICK / data model
 - 84.6 BACnet
 - 84.7 Faults (check-engine)
 - 84.8 Timeseries & sites
 - 84.9 Building & host
 - 84.10 Local agent (Ollama)
 - 84.11 Modbus
 - 84.12 PyPI engine API (separate)
 - 84.13 Errors
- 85 Python package

- 86 Python package (open-fdd)
 - 86.1 Modules
 - 86.2 When to use package-only
 - 86.3 Versioning
 - 86.4 Offline example
- 87 Configuration reference
- 88 Configuration reference
 - 88.1 Bridge / auth
 - 88.2 BACnet
 - 88.3 Docker edge
- 89 Glossary
- 90 Glossary
- 91 Appendix
- 92 Appendix
- 93 Edge Docker deploy
- 94 Edge Docker deploy
- 95 Arrow-native runtime
- 96 Arrow-native runtime
 - 96.1 Authoring contract
 - 96.2 Package layout

Generated June 6, 2026

1 Home

2 Open-FDD

Open-FDD is an open-source platform for **fault detection**, **BACnet integration**, and **operator analytics** at the building edge. It combines a React dashboard, FastAPI Operator Bridge, Python Rule Lab, local feather historian, and optional Brick/RDF modeling.

Who it is for

Audience	Start here
IT / controls engineer trying the stack	Quick Start — Docker
Developer contributing or customizing	Developer Guide
Integrator wiring APIs	Appendix — API routes
Analyst authoring FDD rules	Rule Cookbook · Fault Codes

2.1 What you get

Component	Purpose
Operator Bridge	Web API + dashboard (trends, faults, Rule Lab, model tools)
BACnet tools	Discover, read, poll, optional supervised writes
Rule Lab	Arrow-native apply_faults_arrow(table, cfg) rules on feather historian data
Local historian	Feather-based telemetry store on the edge host
Brick / RDF model	Equipment and point semantics for bindings and analytics
Docker images	Published on GHCR — pull and run without building from source
Python package	pip install open-fdd for offline CSV/notebook and YAML engine use

2.2 Two paths

2.2.1 Try it with Docker (recommended)

Pull published images from ghcr.io/bbartling/, configure auth and BACnet env files, start the stack, open the dashboard.

→ [Quick Start](#)

2.2.2 Develop locally

Clone the repo, create workspace/auth.env.local, build images, run tests, submit a PR.

→ [Developer Guide](#)

2.3 Distribution

Channel	Use when
	Production or trial edge deploy

Channel	Use when
GHCR images ghcr.io/bbartling/openfdd-*	
This repository	Custom builds, BACnet commissioning, Rule Lab development
PyPI open-fdd	Library-only pandas/YAML workflows (no full UI)

2.4 License

MIT — see repository LICENSE.

3 Run with Docker images

4 Run with Docker images

4.1 Images (GHCR)

Image	Role
ghcr.io/bbartling/openfdd-bridge	FastAPI + React dashboard
ghcr.io/bbartling/openfdd-commission	BACnet discover / read / write agent
ghcr.io/bbartling/openfdd-bacnet-poll	Historian poll driver (host network)
ghcr.io/bbartling/openfdd-mcp-rag	Optional doc-search sidecar

Check [GitHub Packages](#) for current tags (e.g. 2026.06.07-edge).

4.2 Option A — Ansible deploy (edge VM)

From your **control machine** (laptop or CI runner with SSH to the edge host):

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd

# One-time: build dashboard assets if not in the image tag you use
./scripts/build_operator_dashboard.sh prod
```

```

# Set image tag from a published GHCR release
export OPENFDD_IMAGE_TAG=2026.06.07-edge

# Deploy (requires inventory + host_vars – see infra/ansible/
inventory.example.yml)
cd infra/ansible
export SSHPASS='your-ssh-password' # or use SSH keys
./deploy.sh docker --limit <inventory_host>

```

Host vars should set `openfdd_docker_pull_from_ghcr: true`. Site BACnet bind, model, and rules sync from `infra/ansible` templates and `edge_backup/local/<site>/`.

Bootstrap helper (docker → Caddy → optional AI → ops check):

```

OPENFDD_IMAGE_TAG=2026.06.07-edge ./scripts/
bootstrap_edge_ghcr.sh --limit <inventory_host>

```

4.3 Option B — Local trial (compose on one machine)

For a quick lab trial without Ansible:

```

git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local # edit
passwords
./scripts/docker_build.sh # or pull GHCR tags into compose via
env
./scripts/openfdd_stack.sh up

```

Open `http://127.0.0.1:8765/` (bridge) or `http://127.0.0.1/` if Caddy is installed locally.

4.4 Required configuration

File	Purpose
<code>workspace/auth.env.local</code>	Operator / integrator login and <code>OFDD_AUTH_SECRET</code>
<code>workspace/bacnet/commissioning/commission.env</code>	BACnet bind IP, instance, discover range
<code>workspace/data/</code>	Bind-mounted historian, rules, model (created on first run)

Copy from `*.example` files in the repo. Never commit real passwords.

4.5 Start / stop

```
# Ansible edge: re-run deploy or on host:
cd ~/open-fdd/docker && docker compose up -d

# Local dev:
./scripts/openfdd_stack.sh up
./scripts/openfdd_stack.sh down
```

4.6 After first deploy

→ First login and health check
→ Updating the stack — Ansible from control machine
→ Live site update (SSH) — tag bump on minimal ~/open-fdd/
folder

5 First login and health check

6 First login and health check

6.1 Health endpoints

URL	Auth	Meaning
GET /health	Public	Bridge process alive
GET /health/ stack	Bearer token	Container traffic-light summary

```
curl -s http://<host>:8765/health | jq .
# or through Caddy:
curl -s http://<host>/health | jq .
```

Stack script (from repo on control machine or edge):

```
./scripts/stack_health_check.sh <edge-ip-or-hostname>
```

6.2 Login

1. Open the dashboard URL (Caddy :80 → bridge :8765).
2. Sign in with credentials from workspace/auth.env.local (default roles: integrator, operator).

3. Confirm **Building insight** / home loads without API errors.

```
curl -s -X POST http://<host>/api/auth/login \
-H 'Content-Type: application/json' \
-d '{"username":"integrator","password":"<from-auth-env>"}'
```

6.3 Smoke checklist

Check	Pass criteria
Dashboard HTML	200 on /
Auth	POST /api/auth/login returns token
Model	GET /api/model/health shows point count
BACnet	Commission container running; discover returns devices (if OT LAN attached)
Historian	Feather store growing under workspace/data/feather_store/
Rule Lab	Lint + test-rule APIs respond (integrator role)

6.4 Troubleshooting

Symptom	Likely fix
401 on all routes	Set <code>auth.env.local</code> ; disable <code>OFDD_AUTH_DISABLED</code> on LAN
Empty BACnet discover	Fix <code>BACNET_BIND</code> in <code>commission.env</code> — see BACnet network setup
Caddy shows welcome page	Reload Caddy config pointing to bridge :8765
Stale UI after upgrade	Hard refresh; confirm new bridge image tag deployed

7 Quick Start

8 Quick Start

Run Open-FDD on a Linux edge host or VM using **published Docker images**. No local image build required.

Step	Page
1	Run with Docker images
2	First login and health check
3	Updating the stack — Ansible or SSH paths
—	Live site update (SSH) — minimal VM folder, no git pull

Prerequisites: Linux host with Docker and Docker Compose, network access to BACnet devices if polling OT equipment, and a safe lab or maintenance window for first BACnet discovery.

BACnet writes are disabled by default. Do not enable write paths on live equipment without an allowlist and operator authorization. See [Write safety](#).

9 Updating the stack

10 Updating the stack

Two paths depending on how you manage the edge host.

10.1 Path A — SSH on the live VM (minimal deploy folder)

Use when the host has only `~/open-fdd/{docker,docker-compose.yml,workspace}` and **no** full git checkout.

→ **Live site update (SSH)** — backup with `sudo tar`, verify GHCR tags, edit `docker-compose.yml`, `docker compose pull + up -d --force-recreate`, smoke checks, rollback.

Then run the control-machine insurance check:

```
cd infra/ansible
./scripts/post_deploy_check.sh --limit <inventory_host>
```

10.2 Path B — Ansible from control machine (inventory)

Dashboard/UI changes need `deploy.sh ui` (or `upgrade_edge_full.sh`), not `image pull` alone. The edge bind-mount `workspace/api/static/app/` overrides image-baked React assets.

Full upgrade (UI + containers + post-deploy check):

```
./scripts/build_operator_dashboard.sh prod # if not already built
OPENFDD_IMAGE_TAG=<new-tag> ./scripts/upgrade_edge_full.sh --limit <inventory_host>
```

Image-only (API/commission/MCP code from GHCR, **same old UI** unless you also ran `ui`):

```
export OPENFDD_IMAGE_TAG=<new-tag>
RUN_POST_CHECK=1 ./scripts/upgrade_edge_ghcr.sh --limit <inventory_host>
```

Image-only sets `openfdd_docker_sync_workspace_data=false` so bind-mounted `~/open-fdd/workspace/data/` is not replaced.

10.3 Full redeploy

When you need to sync site model, rules, or commission files from Ansible backups:

```
OPENFDD_IMAGE_TAG=<tag> ./deploy.sh docker --limit <inventory_host>
```

10.4 Verify after upgrade

Control machine:

```
cd infra/ansible
./scripts/post_deploy_check.sh --limit <inventory_host>
```

On edge host:

```
ls -la ~/open-fdd/workspace/data/feather_store/ # size should not drop to zero
docker compose -f ~/open-fdd/docker-compose.yml ps
curl -sf http://127.0.0.1:8765/health
```

10.5 Rollback

- **SSH path:** restore previous image tag in `docker-compose.yml` and recreate containers — see [rollback section](#).
- **Ansible path:** re-deploy the previous known-good `OPENFDD_IMAGE_TAG`.

Workspace data on the host survives image-only upgrades; keep backups of workspace/ before major config changes.

11 Local development

12 Local development

12.1 Clone and auth

```
git clone https://github.com/bbartling/open-fdd.git && cd open-fdd
cp workspace/auth.env.example workspace/auth.env.local
# Edit integrator/operator passwords and OFDD_AUTH_SECRET (32+
chars)
```

12.2 Start stack

```
./scripts/build_and_test.sh           # dashboard build + pytest
./scripts/docker_build.sh
./scripts/openfdd_stack.sh up
./scripts/stack_health_check.sh
```

URL	Service
http://127.0.0.1:8765	Bridge (direct)
http://127.0.0.1:8765/docs	OpenAPI

12.3 BACnet bench overlay (optional)

```
docker compose -f docker/compose.dev.yml -f docker/
compose.bench.yml up -d
# commission.env – set BACNET_BIND to your OT interface
```

12.4 Docs preview

```
cd docs && bundle install && bundle exec jekyll serve  
# http://127.0.0.1:4000/open-fdd/
```

13 Build Docker images

14 Build Docker images

14.1 Local build

```
./scripts/docker_build.sh  
# Optional tar for air-gap:  
./scripts/docker_build.sh --save
```

Images: openfdd-bridge, openfdd-commission, openfdd-bacnet-poll, openfdd-mcp-rag.

14.2 Publish to GHCR (maintainers)

GitHub Actions workflow **Publish Docker addons** — manual dispatch with a version tag.

Maintainer checklist:

1. Green CI on master
2. Run workflow with tag e.g. 2026.06.06-edge
3. Verify packages at ghcr.io/bbartling/openfdd-*
4. Deploy to a demo edge with `OPENFDD_IMAGE_TAG=<tag>`

Details live in `.github/workflows/` and `scripts/docker_build.sh` — not duplicated here.

15 Developer Guide

16 Developer Guide

Clone, build, test, and contribute to Open-FDD.

Topic	Page
Local setup	Local development
Docker builds	Build Docker images
Tests & CI	Tests and CI
PRs	Contributing

AI-assisted contributors: see repository AGENTS.md for workspace layout, skill routing, and agent policy (one paragraph in public docs; details stay internal).

17 Tests and CI

18 Tests and CI

18.1 Local tests

```
./scripts/build_and_test.sh           # UI + workspace bridge
pytest
python3 -m pytest open_fdd/tests -q   # PyPI package tests
cd workspace/dashboard && npm test    # dashboard unit tests (if
configured)
```

18.2 CI (GitHub Actions)

Workflow	Trigger	Checks
ci.yml	PR / push	pytest, bridge security audit, dashboard build, Jekyll docs
docs-pages.yml	push master	GitHub Pages site
publish-open-fdd.yml	tag open-fdd-v*	PyPI wheel
publish-docker-addons.yml	manual	GHCR images

18.3 Docs build (same as CI)

```
cd docs && bundle exec jekyll build -d /tmp/open-fdd-site
```

Fix any template or link errors before opening a PR that touches docs/.

19 Contributing

20 Contributing

20.1 Expectations

- Open an issue or discuss large changes before big refactors.
- Keep PRs focused; include test or docs updates when behavior changes.
- Do not commit secrets (auth.env.local, Ansible secrets/*.env.local, commissioning CSVs with real site data).
- Follow existing code style in workspace/api/ and workspace/dashboard/.

20.2 Pull request checklist

☐

Tests pass locally (./scripts/build_and_test.sh or scoped pytest)

☐

Docs updated if user-facing behavior changed

☐

docs/ Jekyll build succeeds

☐

No credentials in diff

20.3 Repository layout (short)

Path	Contents
workspace/api/	Operator Bridge (FastAPI)
workspace/dashboard/	React SPA
open_fdd/	PyPI package (engine, playground, reports)
docker/	Compose files and image contexts
infra/ansible/	Edge deploy playbooks
docs/	This site (Jekyll + Just the Docs)

20.4 Security issues

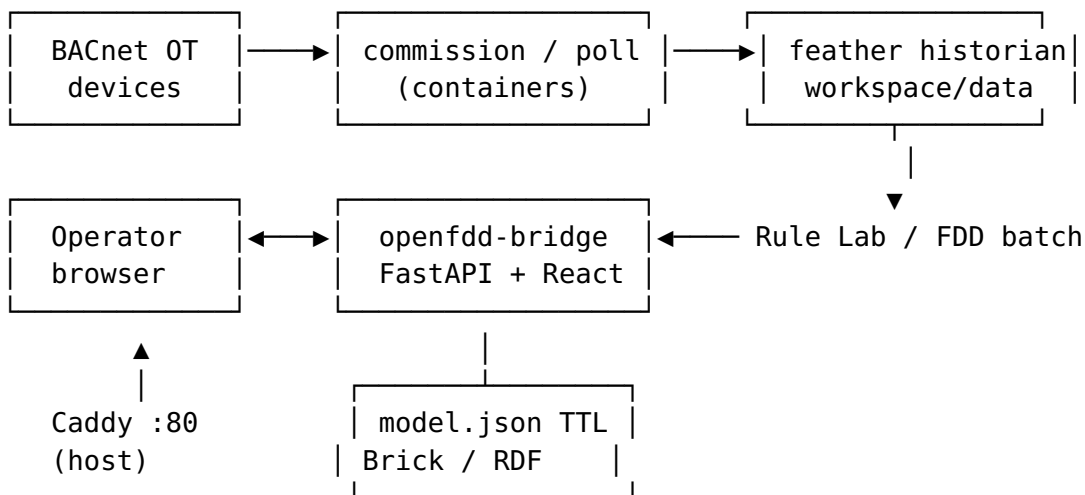
Do not file public issues for vulnerabilities. Contact the maintainer or use GitHub private security advisories.

20.5 Agent-assisted development

Cursor/Claude contributors: read AGENTS.md and skills/ for automation boundaries. Public docs intentionally avoid agent playbook detail.

21 System overview

22 System overview



22.1 Components

Layer	Responsibility
Operator Bridge	Auth, REST/WebSocket API, compiled dashboard, Rule Lab
BACnet commission	Who-Is, read property, supervised writes (gated)
BACnet poll	Scheduled RPM reads → historian
Historian	Feather files per point; local-first retention
FDD runner	Executes workspace/data/rules_py/*.py on schedules or API batch

Layer	Responsibility
Model	Brick TTL + model.json bindings (fdd_input, equipment tree)
MCP RAG	Optional local doc retrieval for agent tools
Caddy	Host reverse proxy :80 → bridge (typical edge)
Ollama	Optional host or container LLM for building insight (not required for FDD)

22.2 Optional PyPI-only path

`pip install "open-fdd[engine]"` runs the YAML/pandas engine; `pip install open-fdd` covers the playground sandbox **without** the Operator Bridge UI. Use for notebooks, CSV backfills, or cloud lambdas. See [Appendix — Python package](#).

23 Arrow recipes

24 Arrow recipes (default)

Open-FDD 3.0 Rule Lab rules use `apply_faults_arrow(table, cfg, context)` and `pyarrow.compute`.

24.1 Simple threshold

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

24.2 Supply air temp high

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    col = str(cfg.get("column", "SAT"))
    return pc.greater(table[col], float(cfg["high"]))
```

Config: {"column": "SAT", "high": 75}

24.3 Fan commanded but no airflow

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    fan_on = pc.greater(table["fan_cmd"],
cfg.get("fan_on_threshold", 0.5))
    low_airflow = pc.less(table["airflow_cfm"],
cfg["min_airflow_cfm"])
    return pc.and_(fan_on, low_airflow)
```

24.4 Economizer not opening

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    oat_good = pc.less(table["outside_air_temp"],
cfg["economizer_oat_limit"])
    cooling = pc.greater(table["cooling_cmd"],
cfg.get("cooling_threshold", 0.5))
    damper_low = pc.less(table["oa_damper_cmd"],
cfg["min_oa_damper_cmd"])
    return pc.and_(pc.and_(oat_good, cooling), damper_low)
```

24.5 Heating and cooling simultaneously

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    heating = pc.greater(table["heat_cmd"],
cfg.get("heat_threshold", 0.1))
    cooling = pc.greater(table["cool_cmd"],
cfg.get("cool_threshold", 0.1))
    return pc.and_(heating, cooling)
```

24.6 Sensor out of range

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    too_low = pc.less(table["sensor_value"], cfg["min_value"])
    too_high = pc.greater(table["sensor_value"], cfg["max_value"])
    return pc.or_(too_low, too_high)
```

More templates ship in `open_fdd.playground.arrow_templates` and via GET `/api/playground/arrow-templates`.

25 Data flow

26 Data flow

26.1 Telemetry ingest

1. **Poll driver** reads BACnet points on a schedule (host-network container).
2. Samples land in the **feather store** (`workspace/data/feather_store/`).
3. Point registry (`points.csv`, discovered inventory) ties BACnet objects to `series_id`.

26.2 Rule evaluation

1. Rules are Python files in `workspace/data/rules_py/` with metadata in `rules_store.json`.
2. **Bindings** map rules to points via the Brick model (`fdd_input` tags).
3. Batch runner (`POST /api/rules/batch` or host timer) produces `fdd_results.json`.
4. Dashboard **faults** view and check-engine status read aggregated results.

26.3 Model sync

- Commissioning imports `update_model.json` / TTL graph.
- `POST /api/model/sync-ttl` refreshes SPARQL-backed tree used by the UI.

26.4 Retention

Feather files grow on disk; retention policies are configurable on the edge. Image upgrades should **not** delete `workspace/data/` when using `upgrade_edge_ghcr.sh`.

27 Python recipes (Arrow)

28 Python recipes (Arrow)

All Rule Lab Python rules use `apply_faults_arrow(table, cfg, context)` on PyArrow tables with `pyarrow.compute`.

See [Arrow recipes](#) for the canonical cookbook (flatline, spread, OOB, schedule faults).

Shared helpers live in `open_fdd.arrow_runtime.cookbook` and `open_fdd.arrow_runtime.windows`.

```
from open_fdd.arrow_runtime.cookbook import flatline_1h_mask

def apply_faults_arrow(table, cfg, context=None):
    return flatline_1h_mask(table, cfg)
```

29 YAML recipes

30 YAML recipes

For **offline** pandas workflows with `open_fdd.engine.RuleRunner` (no Operator Bridge).

```
pip install "open-fdd[engine]"
```

30.1 Minimal rule file

```
id: zone_high_temp
type: threshold
column: zone_temp
params:
  upper: 80
  lower: 65

from open_fdd.engine import RuleRunner
import pandas as pd

df = pd.read_csv("telemetry.csv")
runner = RuleRunner.from_yaml("rules/zone_high_temp.yaml")
flags = runner.run(df)
```

30.2 When to use YAML vs Python Rule Lab

YAML engine	Python Rule Lab
CSV/backfill notebooks	Live edge + dashboard
Fixed rule types in engine	Arbitrary evaluate() logic
CI on historical extracts	BACnet-bound production rules

Rule type reference and column maps: [Appendix — Python package](#).

31 Containers

32 Containers

Container	Network	Persistent mounts
openfdd-bridge	bridge	workspace/ (data, api static, auth env)
openfdd-commission	host or bridge	workspace/bacnet/
openfdd-bacnet-poll	host	workspace/data/, poll profiles
openfdd-mcp-rag	bridge	read-only docs index

Host services (typical edge): Caddy, optional Ollama, openfdd-fdd-loop.timer (docker exec batch).

State lives on the **host filesystem** under workspace/ — containers are replaceable. Prefer GHCR image upgrades over re-provisioning the VM when possible.

33 Architecture

34 Architecture

High-level view of the Open-FDD edge stack.

Page	Topic
System overview	Components and responsibilities
Data flow	Telemetry → historian → FDD → dashboard
Containers	Docker services and host boundaries

35 Rule Cookbook

36 Rule Cookbook

Practical patterns for **Arrow-native Rule Lab**
(`apply_faults_arrow`) on feather historian PyArrow tables.

Page	Use when
Arrow recipes	Default — thresholds, flatline, spread, OOB, fan/schedule faults
Python recipes	Same Arrow patterns with shared <code>open_fdd.arrow_runtime.cookbook</code> imports
Windowing & debugging	Rolling windows, batch runtime, Rule Lab console
YAML recipes	Optional offline <code>open-fdd[engine]</code> workflows (no Operator Bridge)

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

37 Windowing & debugging

38 Windowing & debugging

38.1 Rolling windows (Arrow)

Use `open_fdd.arrow_runtime.windows` for sample-based rolling min/max and consecutive-true streaks:

```
from open_fdd.arrow_runtime.windows import arrow_rolling_min,  
arrow_rolling_max, arrow_consecutive_true
```

Cookbook masks (flatline_1h_mask, spread_1h_mask, oob_mask)
default to ~12 samples (~1 h at 5 min poll).

Config keys: flatline_window_samples, flatline_tolerance,
max_spread, rolling_avg_minutes, bounds_low / bounds_high.

38.2 Rule Lab console

Quick-test and batch responses include backend: arrow, ms, and
flagged row counts. Arrow summary events appear in the event
console on fault hits.

39 Dashboard overview

40 Dashboard overview

40.1 Primary areas

Area	Purpose
Home / Building insight	Check-engine status (green/yellow/red), active fault summary
Trends	Point history from feather store
Faults	Rule hits grouped by equipment family
Rule Lab	Edit, lint, and test Python rules
Data model	Equipment tree, point bindings, import health
BACnet	Discovery, reads, commission workflows
Settings / health	Stack status, auth mode

40.2 Live updates

Dashboard tiles use REST polling and WS /ws/dashboard for
selected live views (ticket auth via POST /api/auth/ws-ticket).

40.3 Roles

Read-only operators see trends and faults; integrators access Rule Lab and model import. See [Auth and roles](#).

41 Auth and roles

42 Auth and roles

42.1 Production (LAN / edge)

- Set `OFDD_AUTH_SECRET` and role passwords in `workspace/auth.env.local`.
- Bridge on non-loopback addresses **requires** auth unless explicit insecure dev flags are set (lab only).

Role	Typical access
viewer	Read trends, faults, model tree
operator	Run batch FDD, acknowledge views
integrator	Rule Lab, bindings, model import
commission	BACnet writes (plus write guard)
agent	Agent tools / app-edit gates
admin	Full configuration

Login: `POST /api/auth/login` → Bearer JWT on API calls.

42.2 Localhost dev

`OFDD_BRIDGE_HOST=127.0.0.1` with `OFDD_AUTH_DISABLED=1` is acceptable on a single machine. **Never** use auth-disabled mode on a building LAN.

Full hardening: [Security](#).

43 Rule Lab

44 Rule Lab

Rule Lab authors **Arrow-native** Python rules against the feather historian. Rules define:

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

The bridge detects `apply_faults_arrow`, runs rules on **PyArrow tables** via `open_fdd.arrow_runtime`, and persists .py sources under `workspace/data/rules_py/`.

- **Quick test** — POST `/api/playground/test-rule` (returns backend: arrow, ms, flagged count)
- **Batch** — POST `/api/rules/batch / openfdd-fdd-loop timer`
- **Templates** — GET `/api/playground/arrow-templates`

Pin points to rules via **Model & assignments** (`/model`) commissioning JSON or BACnet tree right-click.

45 Model workflow

46 Model workflow

46.1 Brick model

- **Equipment tree** — SPARQL over synced TTL (`workspace/data/data_model.ttl`)
- **Point registry** — BACnet inventory → `series_id` for historian bindings
- **Health score** — orphan points, missing `fdd_input`, stale telemetry

46.2 Typical integrator flow

1. Import or sync BACnet point registry.
2. Map equipment classes (AHU, VAV, ...) in the model UI.

3. Tag points with `fdd_input` roles (zone temp, SAT, damper cmd, ...).
4. Bind Rule Lab rules to those inputs.
5. Run `POST /api/model/sync-ttl` after bulk imports.

Export: `GET /api/model/export` (TTL/JSON). API detail in [Appendix](#).

47 Operator Bridge

48 Operator Bridge

The Operator Bridge is the FastAPI service and React dashboard operators use daily.

Page	Topic
Dashboard overview	Main UI areas
Auth and roles	Login and permissions
Rule Lab	Python rule authoring
Model workflow	Brick bindings and imports

REST details: [Appendix — API routes](#).

49 Network setup

50 Network setup

50.1 Bind address

Commission and poll containers need the **OT interface** IP and **UDP 47808**:

```
# workspace/bacnet/commissioning/commission.env
BACNET_BIND=192.168.1.50/24:47808
BACNET_INSTANCE=599999
```

- Use the NIC that can reach controllers (not only the management VLAN unless routed).
- Poll driver uses `network_mode`: `host` on Linux edges.

50.2 Discovery range

```
DISCOVER_LOW=1
DISCOVER_HIGH=4194302
DISCOVER_TIMEOUT=30
```

Narrow the range in large sites to avoid broadcast storms.

50.3 Verify

From commission container logs or API: Who-Is → I-Am responses. If zero devices, check firewall, bind IP, and VLAN routing before tuning rules.

51 Discover and read

52 Discover and read

The **commission** container exposes HTTP APIs for discovery and read-property workflows used by the dashboard BACnet tools.

Capability	Notes
Who-Is / I-Am	Device discovery on OT subnet
Read property	Present value, object name, units
RPM	Bulk reads for inventory export
Write property	Gated — see Write safety

Exported inventory feeds `points_discovered.csv` and the Brick point registry.

52.1 Operator workflow

1. Confirm [network setup](#).
2. Run discover from dashboard or API.
3. Review discovered objects; enable points for poll profiles.
4. Sync model bindings for FDD inputs.

53 Polling

54 Polling

The **bacnet-poll** container runs scheduled reads defined in `device_poll_profiles.csv` and enabled points in `points.csv`.

Setting	Location
Poll profiles	workspace/bacnet/polls/
Enabled points	workspace/data/points.csv
Output	workspace/data/feather_store/

54.1 Enable on edge

Host var `enable_bacnet_poll_driver: true` in Ansible (or compose profile). Driver requires host networking on Linux.

54.2 Health

- Dashboard poll status shows last cycle time.
- Stale points trigger data-quality fault patterns — see [Sensor quality faults](#).

55 Write safety

56 Write safety

BACnet writes can change live plant behavior. Open-FDD disables writes by default. Enable only with operator sign-off, a tested allowlist, and audit logging.

56.1 Defaults

Control	Default
Supervisory writes	Off
Write allowlist	Empty — must enumerate device/object

Control	Default
Audit	Commands logged when writes enabled

56.2 Enable writes (commission role)

1. Set env flag documented in [Security](#) (OFDD_BACNET_WRITE_ENABLED or site equivalent).
2. Populate write allowlist CSV / config with explicit object IDs.
3. Use **commission** role JWT; every write should include reason/ note where API supports it.
4. Test on bench hardware before production OT.

56.3 Operator rules

- Never write setpoints or overrides on life-safety or fire/smoke interlocks through experimental rules.
- Prefer read-only polling for FDD until rules are validated in Rule Lab.
- Revert overrides through BACnet priority relinquish where supported.

57 BACnet

58 BACnet

Open-FDD integrates with field controllers via BACnet/IP (and bench MSTP overlays in dev compose).

Page	Topic
Network setup	Bind address and NIC
Discover and read	Commission agent
Polling	Historian driver
Write safety	Defaults and allowlists

59 Convention

60 Fault code convention

60.1 Format

FAMILY-SUFFIX — suffix is **1-3 letters** (e.g. VAV-C, AHU-B). Numeric suffixes like VAV-03 are avoided (collision with physical equipment names).

60.2 Categories

Category	Meaning
performance_degradation	Efficiency or capacity drift
simultaneous_heat_cool	Heating and cooling fighting
sensor_fault	Flatline, OOB, inconsistent readings
io_fault	Command vs feedback mismatch

60.3 Severity

Level	Operator response
info	Log; trend for maintenance window
warning	Investigate within days
critical	Prompt attention; may affect comfort/energy significantly

60.4 Linking rules

In Rule Lab, set `fault_code` on the rule metadata. Batch FDD aggregates hits into `GET /api/faults/status` by family.

60.5 Cookbook mapping

Each catalog entry lists `cookbook_patterns` (e.g. `flatline_1h`) — see [Rule Cookbook](#).

61 AHU faults

62 AHU faults

Air handling unit codes (sample from catalog).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
AHU-A	Supply fan performance degradation	warning	Spread/runtime vs baseline	Belt slip, dirty filters, VFD limit	Trend speed vs airflow; filter dP
AHU-B	Simultaneous heating and cooling	critical	Heat + cool outputs active together	Sequencing bug, leaking valve	Trend coil commands; valve feedback
AHU-C	SAT sensor fault	warning	Flatline / OOB SAT	Failed sensor, bad calibration	SAT trend; compare to coil discharge
AHU-D	MAT inconsistent with OAT/RAT	warning	MAT outside OAT-RAT envelope	MAT/OAT/RAT sensor errors	Verify damper positions; cross-check OAT
AHU-E	Economizer not economizing	warning	Free cool available, OA minimum	Stuck damper, lockout	OA damper vs OAT trend
AHU-F	Damper command vs feedback mismatch	warning	Cmd $\neq$ feedback	Stuck actuator, linkage	Compare cmd/feedback trends

Example rule: flatline_1h on SAT → tag **AHU-C**. Recipe: [Python recipes](#).

63 VAV faults

64 VAV faults

Variable air volume terminal codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
VAV-A	Zone heating/cooling overlap	warning	Heat and cool at terminal	Reheat valve stuck, SAT too low	Zone temp vs SAT; valve cmd
VAV-B	Poor zone temperature control	warning	Persistent deviation from setpoint	Oversized box, bad tuning	PI loop trends; occupancy
VAV-C	Zone temperature sensor fault	warning	Flatline / OOB zone temp	Failed zone sensor	Flatline test; compare to neighbor zones
VAV-D	Minimum airflow not met	warning	Flow below minimum cmd	Damper stuck, pressure issue	Airflow vs damper cmd
VAV-E	Discharge temp deviation from SAT	warning	DAT far from SAT setpoint	Coil issue, sensor drift	DAT vs SAT SP

Example rule: oob_rolling on zone temp → **VAV-C** or comfort bounds per site policy.

65 RTU faults

66 RTU faults

Packaged rooftop unit codes (sample).

Code	Title	Severity	Detection summary	Likely causes	Operator checks
RTU-A	Compressor short cycling	warning	Rapid on/off cycles	Low charge, oversized unit	Run-time histogram
RTU-B	Economizer / OA damper fault	warning	OA behavior vs conditions	Actuator, sensor	OA damper vs enthalpy/OAT
RTU-C	Supply air temperature fault	warning	SAT flatline or OOB	Sensor failure	SAT trend vs outdoor conditions
RTU-D	Simultaneous heat and cool	critical	Heat + cool stages together	Control board, relay stuck	Stage status trends

Bind rules to packaged unit fdd_input points (SAT, OAT, compressor cmd, ...).

67 Sensor and data quality

68 Sensor and data quality

Code	Title	Severity	Detection summary	Likely causes	Operator checks
BLD-B	Outdoor air sensor fault	warning	OAT flatline / unrealistic	Sensor exposure, failed probe	Compare to weather service
BLD-D	Stale telemetry	warning	No new samples within threshold	Poll driver down, device offline	Poll health; BACnet comms
DC-C	Datacom sensor anomaly	warning	CRAH/ room sensor flatline	Failed sensor	Row-level flatline recipe

68.1 Communication quality

- Monitor poll cycle timestamps in dashboard.
- Use `stale_points` cookbook pattern for dropout detection.
- Do not confuse **comm loss** with **equipment fault** — verify BACnet device online before dispatching mechanical.

68.2 Alarm console quality

When integrating with BMS alarms (future/external), map Open-FDD codes to operator work orders — Open-FDD itself is **analytics-first**, not a dial-out alarm server.

69 Fault Codes

70 Fault Codes

Open-FDD uses a **check-engine** model: one building-level status (green / yellow / red) with a catalog of fixed fault codes per equipment family.

Page	Content
Convention	Naming, severity, categories
AHU faults	Air handling units
VAV faults	Terminal units
RTU faults	Packaged rooftops
Sensor & data quality	Stale, flatline, comms

Live catalog API: `GET /api/faults/catalog` on the Operator Bridge.

71 Live site update (SSH)

72 Live site update (SSH)

Use this runbook when a **live edge VM** already has a minimal deploy folder and you are **not** doing `git pull` on the host.

Typical layout (no full git checkout):

```
/home/bbartling/open-fdd/  
  docker/  
  docker-compose.yml  
  workspace/
```

workspace/ is the **live site state** — BACnet commissioning files, poll CSV, feather historian, BRICK model, FDD rules, MCP RAG index, and env files. Image upgrades must **preserve** it.

72.1 Concepts (plain language)

Term	Meaning
Container	A running app instance (bridge, commission, mcp-rag). Recreated on upgrade.
Image	Downloaded app package from GHCR (e.g. openfdd-bridge:2026.06.07-edge).
Docker volume	Persistent data Docker manages separately. This minimal layout uses bind mounts into workspace/ instead.
workspace/	Open-FDD site state on disk. This is what you must not lose.

Never on live sites: `docker compose down -v`, `docker volume prune`, `docker system prune -a --volumes`, or `docker rm -f $ (docker ps -aq)`. Those can destroy data or leave the site down with no rollback path.

72.2 Before you start

1. Maintenance window or low-traffic period (containers restart briefly).
2. SSH access to the edge host.
3. New GHCR tag published (check [GitHub Packages](#)).
4. Optional: control machine with the full open-fdd repo for post-deploy check.

72.3 1. Backup workspace/

Container processes may write files as **root:root** with mode **0600**. A normal user tar can fail on those paths — use sudo:

```
cd /home/bbartling/open-fdd
```

```

export BACKUP_ROOT="$HOME/openfdd-backups/$(date +%Y%m%d-%H%M%S)"
mkdir -p "$BACKUP_ROOT"

cp docker-compose.yml "$BACKUP_ROOT/docker-compose.yml.before"

docker compose ps > "$BACKUP_ROOT/docker-compose-ps-before.txt"
docker compose config --images > "$BACKUP_ROOT/docker-images-
before.txt"

sudo tar --xattrs --acls -czf "$BACKUP_ROOT/workspace-full.tgz"
workspace
sudo chown "$USER:$USER" "$BACKUP_ROOT/workspace-full.tgz"

echo "Backup saved to: $BACKUP_ROOT"
du -h "$BACKUP_ROOT/workspace-full.tgz"

```

72.4 2. Verify new images exist on GHCR

```

export NEW_TAG="2026.06.07-edge"

docker manifest inspect ghcr.io/bbartling/openfdd-bridge:${
{NEW_TAG}} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-commission:${
{NEW_TAG}} >/dev/null &&
docker manifest inspect ghcr.io/bbartling/openfdd-mcp-rag:${
{NEW_TAG}} >/dev/null &&
echo "All images exist for ${NEW_TAG}"

```

Replace NEW_TAG with the tag you intend to deploy.

72.5 3. Update the React UI bundle (required for dashboard changes)

Image pull alone does not update the browser UI. The bridge serves workspace/api/static/app/ from the **bind mount first**, before anything baked into the GHCR image (static_dashboard_dir() in the bridge). If that folder still has an old index-*.js hash (e.g. index-TRH4YIfA.js), you will keep seeing the old BACnet tree, Host Stats revisions panel, Model & assignments tab, etc.

From bensserver (control machine, full repo):

```

cd /path/to/open-fdd
./scripts/build_operator_dashboard.sh prod

```

```
cd infra/ansible
./deploy.sh ui --limit acme_vm_bbartling
```

Confirm the edge picked up the new hash:

```
curl -sf http://<edge-ip>/ | grep -o 'index-[^"]*\.' | head -1
# Should match bensserver: grep -o 'index-[^"]*\.' workspace/
api/static/app/index.html
```

One command (recommended after CI publishes a new tag):

```
OPENFDD_IMAGE_TAG=2026.06.08-edge ./scripts/upgrade_edge_full.sh
--limit acme_vm_bbartling
```

That runs: dashboard build → deploy.sh ui → GHCR pull + docker compose up -d --force-recreate → post_deploy_check.sh --full.

72.6 4. Update docker-compose.yml and recreate containers

Edit image tags in docker-compose.yml (example: 2026.06.04-edge → 2026.06.07-edge):

```
cd /home/bbartling/open-fdd

cp docker-compose.yml "$BACKUP_ROOT/docker-compose.yml.before-tag-
change"
sed -i "s/2026\.06\.04-edge/${NEW_TAG}/g" docker-compose.yml

docker compose config --images
docker compose pull
docker compose up -d --force-recreate

docker compose ps
```

Bind-mounted workspace/ is unchanged; only container images restart.

72.7 5. On-VM smoke checks

```
cd ~/open-fdd

echo "=== Stack ==="
docker compose ps
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}"

echo "=== Bridge health ==="
curl -sf http://127.0.0.1:8765/health | jq . || curl -sf http://
127.0.0.1:8765/health
```

```

echo "=== Stack health (if available – may require auth) ==="
curl -i http://127.0.0.1:8765/health/stack

echo "=== MCP health from inside mcp-rag container ==="
docker compose exec mcp-rag sh -lc 'curl -sf http://
127.0.0.1:8090/health || wget -qO- http://127.0.0.1:8090/health ||
python - <<PY
import urllib.request
print(urllib.request.urlopen("http://127.0.0.1:8090/health",
timeout=5).read().decode())
PY'

echo "=== BACnet samples still moving ==="
ls -lah workspace/bacnet/polls/samples.csv
tail -n 1 workspace/bacnet/polls/samples.csv

echo "=== BACnet ingest state ==="
sudo cat workspace/data/bacnet_ingest_state.json | jq . || sudo
cat workspace/data/bacnet_ingest_state.json

echo "=== Feather store retained and writing ==="
du -sh workspace/data/feather_store
find workspace/data/feather_store -type f -printf "%TY-%Tm-%Td
%TH:%TM %p\n" | sort | tail -n 10

echo "=== Recent log errors ==="
docker compose logs --since 20m | grep -Ei "error|exception|
traceback|critical|failed|permission|denied" || true

```

/health/stack nuance: /health should return 200 without auth. /health/stack may return 401, 403, 404, or an empty body depending on auth settings — use curl -i (not curl -sf) so you see the status code. A failed silent curl -sf does not mean the stack is broken.

72.7.1 Arrow / FDD rules (Open-FDD 3.0+)

All saved rules use `apply_faults_arrow(table, cfg, context)` on PyArrow tables.

From bensserver:

```
./scripts/validate_fdd_backends.sh --docker # must exit 0
```

On the edge VM after upgrade:

```

grep -R "apply_faults_arrow" -n workspace/data/rules_py 2>/dev/
null | wc -l
docker compose exec bridge python3 -c "
from openfdd_bridge.rule_store import RuleStore
from open_fdd.arrow_runtime.rules import detect_rule_backend
from openfdd_bridge.rule_source import read_source

```

```

    for r in RuleStore().list_rules():
        if not r.get('enabled', True): continue
        code = read_source(r.get('source_path', '')) or
r.get('code', '')
        print(r.get('name'), detect_rule_backend(code, r))
    "

```

Spot-check **Rule Lab** quick-test shows backend: arrow.

72.8 6. Control-machine insurance check

From your laptop or bensserver (full repo clone, not on the edge VM):

```

cd /path/to/open-fdd/infra/ansible

./scripts/post_deploy_check.sh --limit acme_vm_bbartling

# Or explicitly:
./scripts/post_deploy_check.sh --host 100.122.106.124 --ssh-user
bbartling

```

This is the recommended **non-destructive** check: Caddy → React dashboard, /health/stack, MCP, BACnet tree, BRICK/SPARQL, Docker log scan.

Do not run on live commissioned sites:

openfdd_edge_validate.sh --full or acme_operational_verify.sh
— those rediscover BACnet and reset bench model/rules.

72.9 7. Browser validation

Open the site via **Caddy on port 80** (e.g. http://<tailscale-or-lan-ip>/), not only 127.0.0.1:8765:

1. View page source / Network — confirm **new** index-*.js hash (not the pre-upgrade bundle).
2. Home / faults dashboard loads.
3. Integrator login (workspace/auth.env.local).
4. **Host stats** — container image tag / git sha (3.0+ bridge).
5. **BACnet** — right-click point → **Refresh PV, Read priority array**.
6. **Model & assignments** (/model) — commissioning JSON export (404 = bridge image + UI both stale).
7. Rule Lab — quick-test a saved rule (backend: arrow).

72.10 Rollback

If the new tag misbehaves, restore the previous tag in compose and recreate:

```
cd /home/bbartling/open-fdd

# Example: roll back from 2026.06.07-edge to 2026.06.04-edge
export NEW_TAG="2026.06.07-edge"
export OLD_TAG="2026.06.04-edge"

sed -i "s/${NEW_TAG}/${OLD_TAG}/g" docker-compose.yml

docker compose pull
docker compose up -d --force-recreate

docker compose ps
curl -sf http://127.0.0.1:8765/health | jq . || curl -sf http://
127.0.0.1:8765/health
```

workspace/ is unchanged by rollback. For data corruption, extract files from \$BACKUP_ROOT/workspace-full.tgz selectively (model, rules, feather shards) — do not blindly overwrite a running site without stopping containers first.

72.11 Alternative: Ansible image-only upgrade

If you have inventory and SSH from a control machine (no manual sed on the VM):

```
export OPENFDD_IMAGE_TAG=2026.06.07-edge
RUN_POST_CHECK=1 ./scripts/upgrade_edge_ghcr.sh --limit
acme_vm_bbartling
```

See also [Updating the stack](#).

73 LAN hardening

74 LAN hardening

74.1 Deployment modes

Mode	Bind	Auth
Local dev	127.0.0.1	Optional OFDD_AUTH_DISABLED=1
Lab LAN (insecure)	0.0.0.0	Requires explicit insecure dev flags — not production
Edge production	0.0.0.0 behind Caddy	Required credentials + OFDD_AUTH_SECRET

Startup **fails** on non-loopback bind without credentials (unless insecure lab flags are explicitly set).

74.2 Network exposure

- Do not port-forward the bridge to the public internet without TLS and strong auth.
- Keep BACnet on OT VLANs; bridge management on IT VLAN with firewall rules.
- Prefer Caddy on :80/443 rather than exposing :8765 directly.

74.3 Rule Lab caution

Rule Lab executes **operator-authored Python** in a sandbox. Restrict integrator accounts; review rules before enable on production sites.

75 Secrets and auth

76 Secrets and auth

76.1 Files (gitignored)

File	Contents
workspace/ auth.env.local	JWT secret, role passwords
infra/ansible/secrets/ <host>.env.local	SSH deploy secrets (maintainers)

Copy from *.example templates only.

76.2 Required variables (production)

Variable	Purpose
OFDD_AUTH_SECRET	HMAC signing (32+ chars)
OFDD_INTEGRATOR_USER / PASSWORD	Rule Lab, bindings
OFDD_OPERATOR_USER / PASSWORD	Operations

76.3 GHCR pulls

Use read-only registry credentials on edge if images are private; rotate tokens periodically.

76.4 Backup

Back up workspace/data/ and auth env in secure operator vault — not in git.

77 BACnet write allowlists

78 BACnet write allowlists

Supervisory writes require:

- 1. **Feature flag** enabled in environment (off by default).
- 2. **Allowlist** file at workspace/bacnet/write_allowlist.json (copy from `write_allowlist.example.json`). If the flag is on but the allowlist is missing, writes are **denied**.
- 3. **Integrator** role on API calls.
- 4. **Audit** log review after commissioning sessions.

Allowlist entries support `device_instance`, `object_identifier`, `property_identifier`, optional `priority_min/priority_max`, `value_min/value_max`, and `allowed_values` for binary/multistate targets.

Lab-only override: `OFDD_BACNET_WRITE_ALLOW_ANY=1` (audited; not for production edges).

See operator guide: [BACnet write safety](#).

Never enable blanket write access for agent or integrator automation without human approval per site.

79 Operations

80 Operations

Runbooks for live edge hosts after the initial deploy.

Page	When to use
Live site update (SSH)	Minimal <code>~/open-fdd/</code> folder on a VM — pull new GHCR tags, preserve workspace/

For first-time deploy from a control machine, see [Quick Start — Docker](#).

81 Security

82 Security

Concise deployment security for the Operator Bridge and BACnet tools.

Page	Topic
LAN hardening	Auth and bind modes
Secrets and auth	Env files
BACnet write allowlists	Supervisory write gates

83 API routes

84 API routes

REST API served by **openfdd-bridge** (default <http://127.0.0.1:8765>).
Production: Caddy on :80 proxies to the bridge.

OpenAPI: GET /docs and GET /redoc when the bridge runs.

Auth: JWT via POST /api/auth/login. Most routes require
Authorization: Bearer <token>. Roles: viewer, operator, integrator,
commission, agent, admin. BACnet writes require commission role +
[write safety](#). Dev: OFDD_AUTH_DISABLED on loopback only.

WebSocket: WS /ws/dashboard — POST /api/auth/ws-ticket, then
pass the ticket via Sec-WebSocket-Protocol: ofdd.ws, <ticket>.
Query ?ticket= is dev-only when OFDD_WS_ALLOW_QUERY_TICKET=1.
Tickets are short-lived and not interchangeable with the Bearer
token.

84.1 Health & audit

Method	Path	Role	Description
GET	/health	public	Minimal liveness (ok, service, version, auth_required)
GET		auth	

Method	Path	Role	Description
	/health/stack		Stack traffic-light (verbose URLs/bind with OFDD_DEBUG_DIAGNOSTICS)
POST	/api/auth/ws-ticket	auth	Short-lived WebSocket ticket
GET	/api/audit/summary	auth	Audit counters
GET	/api/audit/events	auth	Audit log (query params)
GET	/api/audit/errors	auth	Recent API errors

*Exact public set depends on OFDD_AUTH_DISABLED and middleware.

84.2 Auth

Method	Path	Description
POST	/api/auth/login	Username/password → JWT
GET	/api/auth/me	Current user
GET	/api/auth/status	Auth mode / lockout hints

84.3 Rules & FDD

Method	Path	Role	Description
GET	/api/rules/saved	user	Rule index
POST	/api/rules/save	operator+	Create/update rule metadata
GET	/api/rules/saved/{id}/source	user	Python source
PUT	/api/rules/saved/{id}/source	operator+	Write .py file
GET	/api/rules/assignments	user	Point bindings

Method	Path	Role	Description
POST	/api/rules/ bind	integrator+	Bind rule to point
POST	/api/rules/ bindings	integrator+	Bulk bindings
POST	/api/rules/ batch	operator+	Run all enabled rules → fdd_results.json
GET/POST	/api/rules/ drafts	user	Draft rules

84.4 Rule Lab playground

Method	Path	Description
POST	/api/ playground/ lint	AST lint Python rule
POST	/api/ playground/ test-rule	Run apply_faults_arrow() on feather PyArrow window
POST	/api/ playground/ run-script	Execute script mode
GET	/api/ playground/ sample-frame	Sample DataFrame for editor

84.5 BRICK / data model

Method	Path	Description
GET	/api/model/sites	Site list
GET	/api/model/tree	Equipment tree (SPARQL)
GET	/api/model/graph	RDF graph fragment
GET	/api/model/scope	Scoped query
GET	/api/model/export	Export TTL/JSON
GET	/api/model/health	Model point/equipment counts
GET	/api/model/ttl	TTL metadata
POST	/api/model/sync-ttl	Regenerate TTL from JSON
POST	/api/model/import	Import model payload
POST	/api/model/sites	Create site

Method	Path	Description
GET/ POST	/api/model/bacnet-sync	BACnet ↔ model sync state

84.6 BACnet

Method	Path	Role	Description
GET	/config/bacnet	read	Commission URL / config
GET	/api/bacnet/commission/status	read	Commission agent health
GET	/api/bacnet/server/points	read	Server point table
GET	/api/bacnet/inventory	read	Discovered inventory
POST	/api/bacnet/whois	commission	Who-Is
POST	/api/bacnet/discover	commission	Discover devices
POST	/api/bacnet/read	commission	Read property
POST	/api/bacnet/read-multiple	commission	RPM read
POST	/api/bacnet/write	write	Supervisory write (gated)
POST	/api/bacnet/import-to-model	integrator	Inventory → BRICK
POST	/api/bacnet/driver/sync-discovery	commission	Sync driver tree

Method	Path	Role	Description
POST	/api/bacnet/poll/once	commission	Trigger poll cycle
GET	/api/bacnet/poll/status	read	Poll worker status
POST	/ingest/bacnet	read	Ingest samples into feather

Bind address: BACNET_BIND in commission env — see [BACnet network setup](#). Operator guide: [Discover and read](#).

PATCH | /api/bacnet/driver/point | commission | Enable poll + interval on point |
 PATCH | /api/bacnet/driver/device | commission | Enable poll for all points on device |
 PATCH | /api/bacnet/driver/device/remap | commission | Change instance and/or address |
 DELETE | /api/bacnet/driver/point/{point_id} | commission | Remove point from registry |
 DELETE | /api/bacnet/driver/device/{device_instance} | commission | Remove device |
 DELETE | /api/bacnet/driver/registry | commission | Clear CSVs + model sync |

84.7 Faults (check-engine)

Method	Path	Description
GET	/api/faults/catalog	Fixed fault codes
GET	/api/faults/status	GREEN/YELLOW/RED aggregate
GET	/api/faults/tree	Fault tree by equipment
GET	/api/faults/code/{code}	Code metadata

84.8 Timeseries & sites

Method	Path	Description
GET	/api/timeseries/sites	Feather sites
GET	/api/timeseries/series	Series metadata
GET	/api/timeseries/plot	Plot payload

Method	Path	Description
GET	/api/timeseries/readings	Raw readings
GET	/api/sites	Site list
GET	/api/sites/{site_id}/frame	Wide frame for site

84.9 Building & host

Method	Path	Description
GET	/api/building/status	Building summary
GET	/api/building/alerts	Active alerts
GET	/api/host/stats	CPU/mem/disk

84.10 Local agent (Ollama)

Prefix `/openfdd-agent`. Role `agent` for tool execution.

Method	Path	Description
GET	/openfdd-agent/context	Composed prompt context
GET	/openfdd-agent/tools	Tool manifest
POST	/openfdd-agent/tool	Execute tool (e.g. <code>rules.save</code>)
GET	/openfdd-agent/ollama/health	Ollama reachability
GET	/openfdd-agent/building-insight	Check-engine narrative context
GET	/openfdd-agent/zone-temps	Zone temperature insight
POST	/openfdd-agent/chat	Chat completion (catalog-bound)

Optional host Ollama for building insight — not required for core FDD. Configure via compose profile or host service; see [Architecture overview](#).

84.11 Modbus

Method	Path	Description
POST	/api/modbus/read_registers	Read holding registers

84.12 PyPI engine API (separate)

Library-only HTTP is **not** bundled. See [Python package](#) for RuleRunner and `open_fdd.reports`.

84.13 Errors

JSON error bodies; stack traces hidden unless `OFDD_DEBUG_TRACEBACKS=1`. See [LAN hardening](#).

85 Python package

86 Python package (open-fdd)

Install from PyPI:

```
pip install open-fdd
# Arrow runtime + playground (default 3.0+)
pip install "open-fdd[engine]"           # + YAML RuleRunner (pandas)
pip install "open-fdd[legacy]"          # alias for pandas-backed
legacy helpers
```

86.1 Modules

Module	Purpose
<code>open_fdd.arrow_runtime</code>	Default — <code>apply_faults_arrow</code> on PyArrow Tables
<code>open_fdd.playground</code>	Arrow templates + legacy <code>evaluate()</code> sandbox
<code>open_fdd.engine</code>	Optional YAML RuleRunner on pandas DataFrames
<code>open_fdd.reports</code>	Optional summary/plot helpers

86.2 When to use package-only

Scenario	Use
Notebook on CSV export	engine + YAML rules
Cloud lambda (no UI)	playground.rule_lab
Full building operator stack	Docker Operator Bridge

86.3 Versioning

Publish: `git tag open-fdd-vX.Y.Z` → GitHub Actions **Publish open-fdd**.

```
import open_fdd
print(open_fdd.__version__)
```

86.4 Offline example

```
from open_fdd.engine import RuleRunner
import pandas as pd

df = pd.read_csv("ahu_telemetry.csv")
runner = RuleRunner.from_yaml("rules/high_sat.yaml")
result = runner.run(df)
```

API surface: `open_fdd/engine` docstrings and docs/
`config_schema.json` for YAML rule shapes.

87 Configuration reference

88 Configuration reference

88.1 Bridge / auth

Variable	Default	Notes
OFDD_BRIDGE_HOST	127.0.0.1	0.0.0.0 for LAN
OFDD_AUTH_SECRET	—	Required for LAN
OFDD_AUTH_DISABLED	0	Localhost dev only
OFDD_INSECURE_LAN_DEV	0	Lab only with auth disabled

88.2 BACnet

Variable	Notes
BACNET_BIND	ip/mask:47808 in commission.env
OFDD_BACNET_WRITE_ENABLED	Default off — site-specific name may vary

88.3 Docker edge

Variable	Notes
OPENFDD_IMAGE_TAG	GHCR tag for Ansible deploy
openfdd_docker_pull_from_ghcr	Ansible host_var

Full schema for YAML engine rules: docs/config_schema.json.

89 Glossary

90 Glossary

Term	Definition
Operator Bridge	FastAPI service serving API + static dashboard
Rule Lab	UI and APIs for Arrow apply_faults_arrow() rules
Feather store	On-disk columnar historian under workspace/data/feather_store/
fdd_input	Brick tag linking a point to a rule binding role
Check-engine	Building-level GREEN/YELLOW/RED fault summary
Commission	BACnet discovery/read (and optional write) agent container
Poll driver	Scheduled BACnet RPM reads into historian
GHCR	GitHub Container Registry — ghcr.io/bbartling/openfdd-*

91 Appendix

92 Appendix

Reference material for integrators and library users.

Page	Content
API routes	Operator Bridge REST/WebSocket
Python package	PyPI open-fdd
Configuration reference	Key env variables
Glossary	Terms

93 Edge Docker deploy

94 Edge Docker deploy

Open-FDD edge hosts run three GHCR images (bridge, commission, mcp-rag) with workspace/ bind-mounted for site state.

Phase	Doc
First deploy (Ansible or bootstrap)	Quick Start — Run with Docker images
Live site update (SSH, minimal folder, no git pull)	Operations — Live site update
Image-only upgrade (control machine + inventory)	Quick Start — Updating the stack
Full upgrade (UI static bundle + images)	scripts/ upgrade_edge_full.sh — see Live site update §3

The live VM layout is only docker/, docker-compose.yml, and workspace/ — not a full git checkout. Use the [live site update](#) runbook for tag bumps on that layout.

95 Arrow-native runtime

96 Arrow-native runtime

Open-FDD 3.0 executes Rule Lab rules on **PyArrow tables** end to end.

```
feather_store.read_site_table()
→ fdd_runner / playground.run_arrow_table()
→ apply_faults_arrow(table, cfg, context)
→ pyarrow.compute masks
```

96.1 Authoring contract

```
import pyarrow.compute as pc

def apply_faults_arrow(table, cfg, context=None):
    return pc.greater(table["zone_temp"], cfg["max_zone_temp"])
```

Cookbook helpers: `open_fdd.arrow_runtime.cookbook` (flatline, spread, OOB, after-hours fan).

Script-mode analytics rules receive table (PyArrow) and `cfg` in the sandbox — not pandas DataFrames.

96.2 Package layout

Module	Role
<code>open_fdd.arrow_runtime.backend</code>	Execute rule code, batch chunks
<code>open_fdd.arrow_runtime.cookbook</code>	Shared fault masks
<code>open_fdd.arrow_runtime.windows</code>	Rolling min/max, consecutive-true
<code>open_fdd.playground.arrow_templates</code>	Rule Lab starter templates